

claude-windows / 744270aa-6a38-4011-b018-cdd9aae94849

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\744270aa-6a38-4011-b018-cdd9aae94849\subagents\agent-ab2c45aad7352c9ed.jsonl`
- SHA-256: `a3b8fa381345d17683eda27b304823b600f9ed22e451ff86ae0d895c9c389263`
- Source modified: `2026-07-09T03:26:28+00:00`
- Imported at: `2026-07-09T08:31:56+00:00`
- project: `subagents`
- session_id: `744270aa-6a38-4011-b018-cdd9aae94849`

Transcript

1. user (2026-07-09T03:22:59.781Z)

Read-only exploration of the repo at C:\Users\avidu\Projects\khealsutra-guru\badminton-highlight-indexer. I'm building a standalone, release-gating PERFORMANCE comparison tool and want to integrate it into the EXISTING nightly test framework (which currently does QUALITY/accuracy + reel-count/cost). Map the existing framework precisely so I integrate cleanly rather than duplicate. Do NOT modify any files.

Answer with exact file:line citations and copied snippets:

1. ****Nightly scripts inventory.**** Read `scripts/nightly\_regression.py` and `scripts/nightly\_reelcount.py` fully enough to report, for each: what it does, its CLI/args, its INPUT baselines (paths under `eval\_baselines/` ? e.g. reelcount_manifest.json, reelcount_baseline.json), its OUTPUT convention (dir + filename pattern, e.g. `output/nightly\_reelcount/<date>.{json,md}`, and any `LATEST\_STATUS.txt`), its EXIT CODES / how it signals pass vs regression (gating), and whether it measures timing/cost. Copy the argparse blocks and the exit-code logic.

2. ****Is there a nightly ORCHESTRATOR / wrapper?**** Search for: `scripts/nightly\_regression.ps1`, `scripts/nightly\*.ps1`, `scripts/nightly\*.py` (any other), a `nightly.ps1`/`nightly.sh`, or a Python module that runs several nightly legs and writes a combined status. Grep the repo for `LATEST\_STATUS`, `nightly`, `schtasks`, `Register-ScheduledTask`, `crontab`, `cron`, `.github/workflows` referencing nightly. Report what exists and what's missing. (Context: a sibling STUB repo at C:\Users\avidu\Projects\badminton-highlight-indexer has output/nightly_regression/LATEST_STATUS.txt saying a wrapper `scripts/nightly\_regression.ps1` is "absent, awaiting merge" ? so a scheduler expects that path. Find any reference to this wrapper in THIS repo.)

3. ****Existing accuracy gate.**** Read `run\_regression.py`, `run\_eval.py`, and `backend/eval/regression.py` (the parts about baseline snapshot/compare, tolerance, exit codes). Report the pattern: how a baseline is stored (JSON schema), how `--update-baseline` works, how a regression is detected (tolerance), and the exit-code contract (I saw run_regression.py uses exit 1=regression, 3=no baseline). Copy the key functions' signatures.

4. ****CI wiring.**** Look in `.github/workflows/` and `run\_all\_tests.py`, `pyproject.toml`, `pytest.ini`, `ruff.toml`, `mypy.ini`. Report how tests/lint/type/coverage run in CI (the commands + coverage floor `--cov-fail-under`), and whether any nightly/perf job exists in CI. Copy the relevant CI yaml steps.

5. ****Conventions to match.**** Report the repo's conventions for: (a) where dev/eval scripts live (`scripts/` vs `tools/` ? list what's in each), (b) how `eval\_baselines/` is structured (list files + their JSON shape), (c) how output report dirs are named/dated, (d) how these scripts import backend + load config (do they `load\_config()`? use argparse? emit both `.json` and `.md`?), (e) any shared reporting/util helpers they reuse (e.g. a markdown-report writer, a billing/cost module `backend.billing`, `backend.utils.run\_telemetry`).

6. ****Docs.**** Find any docs describing the nightly/eval/golden-set process:

`docs/GOLDEN\_SET\_IMPLEMENTATION.md`, `docs/GOLDEN\_VIDEOS.md`, README sections on nightly/eval/regression, `docs/ALPHA\_LAUNCH\_READINESS.md`. Summarize what's documented about running these gates and where a new perf-gate doc should live.

Return a structured report with the 6 sections, exact file:line citations, and copied code for: the two nightly scripts' argparse + exit logic, the regression baseline schema + compare function, the CI test/lint steps, and the eval_baselines file shapes. Be concrete ? I'll model the new tool's baseline format, output convention, exit codes, and nightly hook on these.

2. assistant (2026-07-09T03:26:28.742Z)

API Error: 529 Overloaded. This is a server-side issue, usually temporary ? try again in a moment. If it persists, check <https://status.claude.com>.